Dossier de validation technique

Live scan, recherche et Wardrobe Agent

Ce document explique, pour trois fonctions de DressMe, **ce que fait le code**, **pourquoi on peut lui faire confiance** et **où sont ses limites**. Chaque affirmation renvoie à un fichier ou à un test.

Principe commun : **le LLM propose et rédige, le code déterministe vérifie, calcule et écrit**. Aucun LLM n'a d'outil qui écrit dans la base.

Fonction	Où	Type	Ce qu'elle fait
Live scan	BO1 :8000 + page Wardrobe	pipeline de vision + 1 agent (anti-doublon)	ajoute des vêtements en pointant la caméra, sans appuyer sur un bouton
Recherche	BO1 :8000	3 voies : par filtres, par photo, par question libre	retrouve des pièces de la vraie garde-robe
Wardrobe Agent	BO1 :8000, /a2a/wardrobe	agent LangGraph à 4 outils	propriétaire de la garde-robe : cherche, compte, compare, corrige

Comment les trois se relient

- Le **Live scan** utilise le Wardrobe Agent : son agent anti-doublon est la fonction `check_duplicate` du Wardrobe Agent.
- La **recherche** par question libre passe aussi par lui : l'AI Stylist lui délègue la lecture de la garde-robe en A2A.
- Les autres agents (Trip Agent, Gap Agent, Outfit Builder) lisent la garde-robe uniquement par lui.

1. Live scan

Objectif : ajouter un vêtement en le montrant à la caméra. Rien n'est enregistré sans contrôle ; une mauvaise image ou un doublon est signalé à l'utilisateur au lieu d'être sauvegardé.

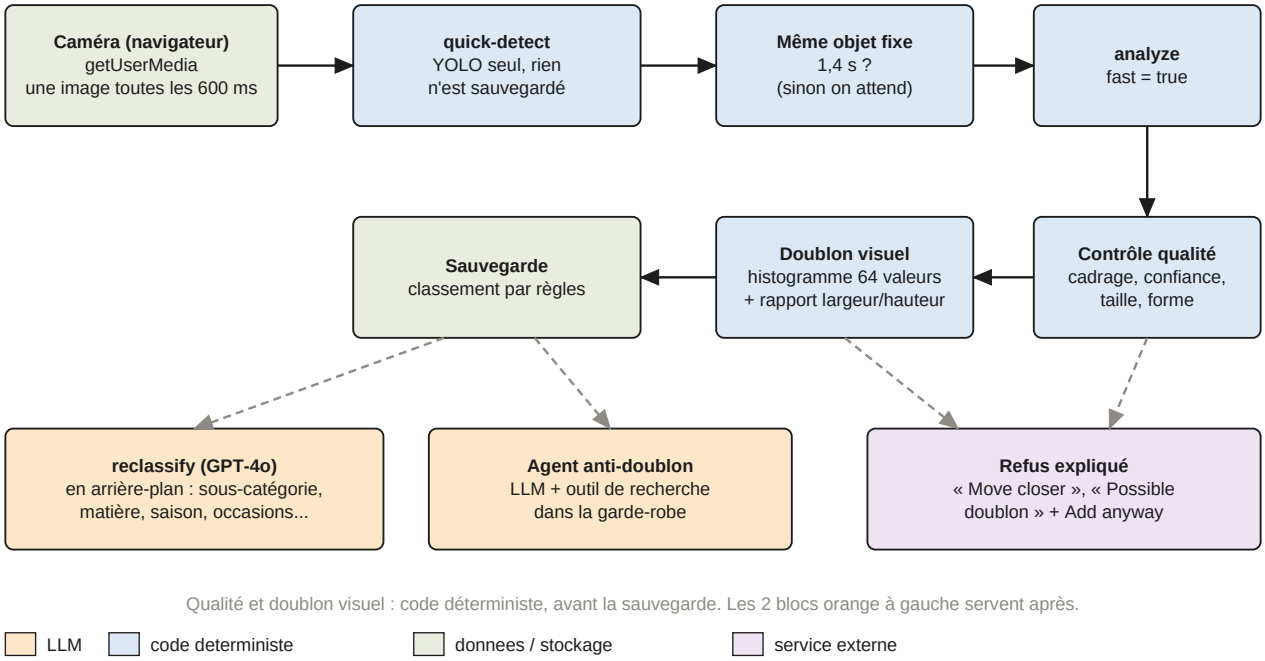

Figure 1 - le chemin d'une capture, de la caméra à la garde-robe.

1.1 Côté page (LiveScan.jsx)

Réglage	Valeur	Rôle
`POLL_MS`	600 ms	fréquence des questions « qu'y a-t-il dans l'image ? »
`STABLE_MS`	1 400 ms	temps pendant lequel le même objet doit rester immobile avant la capture
`COOLDOWN_MS`	2 200 ms	après une capture, on ignore l'image pour laisser changer d'objet
`MIN_CONFIDENCE`	0,35	en dessous : bruit de fond, rien n'est affiché

- Un objet est identifié par *catégorie* + *couleur*. Tant qu'il reste devant la caméra après sa capture, il n'est pas ajouté une seconde fois (« Already scanned »). Il faut l'enlever pour réarmer.
- Un seul échec réseau est ignoré ; trois échecs d'affilée affichent un message à l'utilisateur.
- La caméra arrière est préférée sur téléphone, avec repli sur n'importe quelle caméra.

1.2 Détection (detector.py, YOLO11m-seg)

- Modèle de segmentation entraîné sur les vêtements (`dressme_best.pt`), image redimensionnée à 640 px, seuil de confiance 0,25 ; si rien n'est trouvé, un second essai à 0,05 propose au moins une hypothèse signalée comme peu sûre.
- Le vêtement principal est celui qui maximise *confiance x racine(surface)* : un objet net et grand gagne sur un objet petit en arrière-plan. Deux détections qui se recouvrent de plus de 60 % (IoU) sont fusionnées en une seule.
- La couleur est mesurée sur les pixels du vêtement après érosion du masque, pour ne pas compter les bords ni le fond. Le détourage garde un bord adouci et est limité à 1 024 px.
- L'inférence est protégée par un verrou : un seul calcul à la fois sur le modèle partagé.

1.3 Contrôle qualité (quality.py), avant toute sauvegarde

Contrôle	Seuil	Message à l'utilisateur
Confiance de la détection	< 0,50 (`LIVE_MIN_CONFIDENCE`)	« Not sure what this is. Hold it up against a plain background. »
Vêtement coupé par le bord	boîte à moins de 1,2 % du bord	« Show the whole item »
Trop loin	surface < 4 % de l'image	« Move a little closer »
Forme incomplète	largeur/hauteur au-delà de 1,3 (haut, veste, pull), 1,1 (robe), 0,95 (pantalon)	« Only part of the item is visible »

Les seuils de forme viennent de mesures sur une vraie garde-robe : les mauvaises captures avaient un rapport de 1,4 à 2,5, les bonnes 1,0 ou moins. Pour une photo envoyée depuis la galerie (pas la caméra), seule la confiance est vérifiée : un vêtement qui remplit l'image est normal.

1.4 Détection de doublon, en deux étages

Étage 1, déterministe, avant la sauvegarde. Chaque vêtement a une empreinte : un histogramme de couleurs (4 x 4 x 4 = 64 valeurs, calculé sur les pixels du détourage) et son rapport largeur/hauteur. Le candidat est comparé aux pièces *de la même catégorie* :

```
doublon si intersection d'histogrammes >= 0,80 ET écart de rapport <= 0,20
```

Calibrage mesuré : les vrais doublons font 0,86 à 1,00 avec un écart de forme de 0,00 à 0,19 ; des vêtements noirs différents atteignent 0,97 en couleur mais avec un écart de forme d'au moins 0,24. Les deux conditions ensemble évitent donc de confondre deux pièces noires. Les empreintes des pièces déjà enregistrées sont calculées une fois puis gardées en cache.

Étage 2, l'agent anti-doublon, après la sauvegarde. La page appelle `/api/agent/check-duplicate`. C'est une boucle d'outils : le LLM (température 0,1) dispose d'un seul outil, `search_wardrobe`, jusqu'à 4 tours, et doit répondre en JSON strict `{"duplicate", "similar_item_id", "note"}`. Son prompt dit qu'« un deuxième t-shirt marine » n'est pas un doublon : seule une même pièce vue deux fois l'est. La pièce qui vient d'être sauvegardée est exclue de la comparaison, pour qu'elle ne soit pas son propre doublon.

1.5 Sauvegarde, puis classement en deux temps

- **Immédiat** (`analyze?fast=true`) : classement par règles, sans LLM, pour que la pièce apparaisse en moins d'une seconde.
- **En arrière-plan** (`reclassify`) : GPT-4o reçoit le détourage et les couleurs mesurées, et renvoie une fiche en JSON strict (sous-catégorie, matière, saisons, occasions, formalité, chaleur, description, conseils). La carte se met à jour toute seule.
- Le LLM ne peut changer la catégorie du détecteur qu'en expliquant pourquoi ; ses couleurs sont choisies dans une palette fixe ; ses scores de confiance sont demandés honnêtes.
- Si GPT-4o est absent ou en erreur, on garde le classement par règles : l'ajout n'est jamais bloqué.
- Si l'utilisateur confirme malgré l'alerte (« Add anyway »), la requête repart avec `force` et `only` : seule la détection confirmée est sauvegardée.

2. Recherche

Il y a trois façons de chercher dans la garde-robe. Elles ne coûtent pas la même chose, et seule la dernière utilise un LLM de bout en bout.

Voie	Route	LLM ?	Principe
Par filtres	`A2A data.action=search`	non	filtres exacts sur des valeurs autorisées ; résultat déterministe

Voie	Route	LLM ?	Principe
Par photo	`POST /api/search/photo`	non (YOLO seul)	détecte le vêtement de la photo puis cherche le plus proche dans la garde-robe
Par question libre (page Wardrobe)	`POST /api/wardrobe/agent`	oui (Wardrobe Agent)	le LLM choisit les filtres, le code filtre ; une correction n'est que proposée

2.1 Recherche par filtres (le moteur commun)

La fonction `search()` du Wardrobe Agent est le seul moteur de recherche ; tous les autres agents l'utilisent. Filtres possibles, tous combinés par ET :

Filtre	Comparaison
category, category_group, style, pattern, season, occasion	valeur exacte, prise dans une liste fermée (enum)
color_name	valeur exacte de la palette
min_warmth	chaleur de la pièce >= le minimum demandé (1 à 5)
text	le texte est cherché dans le nom, la sous-catégorie, la matière, la description et les tags

- Résultat limité à 20 pièces par appel, avec seulement 12 champs utiles (id, nom, catégorie, couleur, motif, matière, saisons, style, occasions, formalité, chaleur).
- Comme les filtres sont des listes fermées, un LLM ne peut pas inventer une catégorie : il choisit parmi les valeurs de la taxonomie, sinon l'appel est refusé.
- Le prompt impose d'appeler la recherche **avant** de parler de ce que possède l'utilisateur, et de la relancer avec d'autres filtres si le résultat est vide ou trop large.

2.2 Recherche par photo

- YOLO détecte le vêtement principal ; aucun appel GPT-4 pour rester assez rapide pour « prends une photo pour chercher ».
- Candidats : les pièces de la même catégorie ; s'il n'y en a aucune, tout le groupe (par exemple « haut » puis « pull »).
- Score : 2 points pour la même couleur exacte, 1 point pour la même famille de couleur, 0 sinon ; la meilleure pièce est renvoyée.
- Rien n'est sauvegardé ; la page appelle ensuite l'agent anti-doublon pour dire « vous possédez déjà quelque chose de très proche ».

2.3 Question libre : Wardrobe Agent, et variante AI Stylist

La page Wardrobe envoie la question au Wardrobe Agent (voir la partie 3). La route `/api/agent/search` décrite ci-dessous existe aussi côté serveur, mais aucune page ne l'appelle aujourd'hui : dans l'interface, l'AI Stylist répond par le chat, via l'orchestrateur.

- Entrée de `/api/agent/search` : `message`, la ville et l'unité de température. La garde-robe est lue **par le Wardrobe Agent en A2A**, avec le jeton de l'utilisateur.
- Boucle d'outils GPT-4o (température 0,3, JSON), 8 tours maximum, 3 outils : `ask_wardrobe_agent` (le Wardrobe Agent répond avec ses propres filtres), `get_weather` (Open-Meteo) et `fashion_advice` (livres de mode, avec sources).
- Garde-fous : un rappel unique si le modèle saute la météo ou les livres ; seules les sources réellement citées sont renvoyées ; les pièces de la tenue sont retrouvées par leur nom dans le placard réel ; si la météo ou les livres sont en panne, l'agent le dit et répond sans eux.

3. Wardrobe Agent

Rôle : **propriétaire unique des vêtements de l'utilisateur** (`backend/app/wardrobe_agent.py`). Il lit, compte, compare et corrige. Les autres agents ne lisent pas la base chacun de leur côté.

3.1 Graphe LangGraph

```
START -> gate -+-> apply (une modification attend et tu réponds oui) -> END
               +-> cancel (une modification attend et tu réponds non) -> END
               +-> assist (tout autre message : la boucle d'outils) -> END
```

Outil	Effet	Écrit ?
`search_wardrobe`	filtre la garde-robe (voir 2.1)	non
`find_similar`	pièces possédées proches d'une pièce de référence, classées avec les raisons (calculé par le code)	non
`wardrobe_stats`	comptes par groupe, catégorie, couleur, saison, occasion, et nombre de pièces à vérifier (calculé par le code)	non
`update_item`	**propose** une correction ; rien n'est sauvegardé	non

3.2 Pourquoi une modification ne peut pas être faite par erreur

La protection est dans le code, pas dans le prompt : **le LLM n'a aucun outil qui écrit**.

- **Proposition.** `update_item` vérifie que la pièce existe et que le champ est modifiable, puis mémorise la proposition dans `PendingStore`, clé = (utilisateur, conversation), durée de vie 15 minutes. Le LLM doit alors dire précisément ce qui changerait, et demander confirmation.
- **Confirmation.** Le nœud `gate` regarde si une proposition attend. Un « oui / yes / ok / confirme / vas-y... » (ou `confirm=true`) l'applique ; un « non / annule... » l'annule ; toute autre phrase **supprime** la proposition et traite le message comme une nouvelle demande.
- **Écriture.** Le nœud `apply` passe par `_apply_patch`, le même chemin validé que le bouton de modification de la page. Champs modifiables : nom, catégorie, couleur, motif, matière, style, saisons, occasions, formalité et chaleur (entiers de 1 à 5) ; un nom vide ou un champ inconnu est refusé.
- **Isolation.** Un autre utilisateur, ou une autre conversation, ne peut pas confirmer la proposition de quelqu'un d'autre.
- Si l'écriture est refusée, la réponse est « I could not save the change » avec la raison : jamais « c'est fait » à tort.

3.3 Interface A2A (/a2a/wardrobe)

Requête	LLM ?	Utilisé par
`data.action = list`	non	Trip Agent, Gap Agent : toute la garde-robe
`data.action = search`	non	filtres exacts
`data.action = ask`	oui, lecture seule	AI Stylist, Similar Agent : « une veste chaude pour le travail »
`data.action = check-duplicate`	oui (outil de recherche)	Live scan
texte libre	oui	page Wardrobe et orchestrateur ; une correction passe en *input-required* jusqu'au oui

Six compétences sont publiées sur sa carte : recherche, lecture, correction, statistiques, pièces similaires, détection de doublons. Chaque appel porte le jeton Supabase : l'agent ne voit que les pièces de l'appelant.

4. Preuves et limites

4.1 Ce qui est vérifié par des tests

55 tests passent sur les fichiers concernés (`test_wardrobe_agent.py`, `test_wardrobe_a2a.py`, `test_api.py`, `test_agent_rag.py`, `test_similar_agent.py`).

Affirmation	Test
Une correction n'est que proposée : rien n'est sauvegardé avant le oui	a_correction_is_only_proposed_and_nothing_is_saved_before_the_user_confirms
L'écriture n'a lieu qu'après un oui dans la même conversation (plusieurs formulations)	the_change_is_written_only_after_a_yes_in_the_same_conversation
Un non, une autre demande ou un autre utilisateur n'écrit jamais	no_or_a_different_request_or_another_user_never_writes
Pièce inconnue, champ interdit ou valeur invalide : refusés au LLM	an_unknown_piece_a_forbidden_field_or_an_invalid_value_is_refused_to_the_llm
La recherche renvoie les pièces trouvées et les filtres choisis	a_search_request_returns_the_pieces_the_llm_found_and_the_filters_it_chose
Statistiques et pièces similaires sont calculées par le code	stats_and_similar_pieces_are_computed_by_code
L'agent anti-doublon n'utilise que l'outil de recherche et répond en JSON	the_duplicate_check_of_the_live_scan_uses_the_search_tool_and_answers_in_json
Il ne peut pas appeler la base de connaissances	the_duplicate_check_cannot_call_the_knowledge_base
La même photo deux fois est un doublon tant que l'utilisateur ne confirme pas	same_photo_twice_is_a_duplicate_until_confirmed
Une demande vide demande quoi faire ; un LLM en panne est signalé	an_empty_request_asks_what_to_do_and_a_failing_llm_is_reported

4.2 Limites à dire clairement

- **Doublon visuel** : l'empreinte est un histogramme de couleurs et une forme, pas une vraie reconnaissance d'image. Deux pièces très proches en couleur et en forme (deux t-shirts noirs unis) peuvent être signalées à tort ; l'utilisateur garde la main avec « Add anyway ». Les seuils 0,80 et 0,20 sont calibrés sur une garde-robe réelle, pas sur un grand échantillon.
- **Détection** : le modèle est le point faible sur les fonds chargés, plusieurs vêtements qui se recouvrent ou les accessoires ; en mode caméra, seul le vêtement principal est pris.
- **Classement immédiat** : avant le passage de GPT-4o, la fiche est rudimentaire (règles). Sans clé OpenAI elle le reste.
- **Agent anti-doublon** : sans clé OpenAI, ou si le LLM échoue, il répond « pas de doublon » : l'étage 1 reste la seule protection dans ce cas.
- **Recherche par photo** : le score ne regarde que la couleur ; elle ne compare ni la matière ni la forme.
- **Wardrobe Agent** : les propositions sont gardées en mémoire du processus (15 minutes) ; un redémarrage de BO1 les efface. Les filtres sont exacts : « bleu marine » ne trouve pas « bleu ».
- **Coût** : une demande en texte libre ajoute un appel LLM de plus par saut d'agent ; les routes `list` et `search` n'en ont aucun.

4.3 Fichiers de référence

Sujet	Fichier
Page du Live scan	DressMe UI Blueprint/src/features/wardrobe/LiveScan.jsx
Détection YOLO	backend/app/detector.py
Contrôle qualité et doublon visuel	backend/app/quality.py
Classement GPT-4o et repli	backend/app/enricher.py
Routes (analyze, quick-detect, search/photo, agent/search, check-duplicate)	backend/app/main.py
Wardrobe Agent et agent anti-doublon	backend/app/wardrobe_agent.py
Interface A2A	backend/app/a2a_agents.py, backend/app/wardrobe_a2a.py
AI Stylist (recherche libre)	backend/app/agent.py